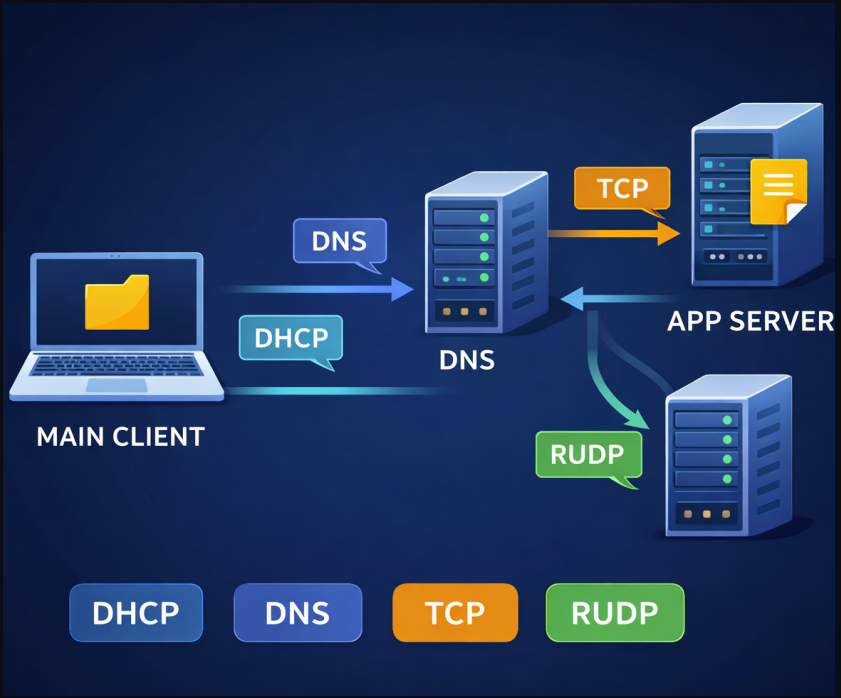
RAMI FADOOL'S PROJECT



פרויקט סוף, רשתות תקשורת תשפ"ו

← </build_future>

Table of contents

dhcp_config.py 01

dhcp_server.py 02

dhcp_client.py 03

dns_config.py 04

dns_server.py 05

dns_client.py 06

app_config.py 07

tcp_server.py 08

Table of contents

File content 09

tcp_client.py 10

rudp_server.py 11

rudp_client.py 12

main_client.py 13-16

ChatGPT prompts 17

State diagram 18

HOW DOES THE SYSTEM HANDLE PACKET LOSS? 19

HOW DOES THE SYSTEM HANDLE LATENCY PROBLEMS? 20

DHCP_CONFIG

In this file I put all the configuration values for DHCP, like server address, port, buffer size, and the IP that the server gives to the client.

I used a separate config file to keep the code clean and make it easy to change values without modifying the main code.

```
DHCP_SERVER_HOST = "127.0.0.1"
DHCP_SERVER_PORT = 5555

# max number of bytes to recieve in one packet
BUFFER_SIZE = 1024

# server offer to client this ip
OFFERED_IP = "192.168.1.100"

# server address that the client gets from DHCP
DNS_SERVER_IP = "192.168.1.2"

SUBNET_MASK = "255.255.255.0"
GATEWAY = "192.168.1.1"
```

DHCP_SERVER

This file implements the DHCP server. Its job is to give network configuration to the client when it connects. The server uses a UDP socket because DHCP works over UDP. It binds to the host and port from the config file and waits for messages from clients.

The server handles two types of messages:

DISCOVER – the client asks for an IP address

REQUEST – the client asks to confirm the IP

When the server receives DISCOVER, it sends an OFFER with the IP, DNS, subnet mask, and gateway.

When the server receives REQUEST, it checks that the client asks for the same IP that was offered, and then sends ACK to approve the configuration.

DISCOVER → OFFER → REQUEST → ACK.

The server runs in a loop so it can keep serving clients.

```python
import socket
import json
from dhcp_config import (
    DHCP_SERVER_HOST,
    DHCP_SERVER_PORT,
    BUFFER_SIZE,
    OFFERED_IP,
    DNS_SERVER_IP,
    SUBNET_MASK,
    GATEWAY
)


def main():
    # create a UDP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # bind the socket to the server address and port from the config file
    server_socket.bind((DHCP_SERVER_HOST, DHCP_SERVER_PORT))

    print("DHCP server started")
    print(f"Listening on {DHCP_SERVER_HOST}:{DHCP_SERVER_PORT}...")

    # this variable keeps the last IP that was offered
    last_offered_ip = None

    while True:
        # wait for a packet from client
        data, client_address = server_socket.recvfrom(BUFFER_SIZE)

        try:
            # decode and convert from JSON text to a dictionary
            message = json.loads(data.decode())
        except json.JSONDecodeError:
            print("Received invalid JSON message")
            continue

        msg_type = message.get("type")
        client_id = message.get("client_id")

        print("Received from client:", message)

        # check client id (to identify and request a specifec IP address)
        if client_id is None:
            print("ERROR: Message doesn't contain client_id")
            continue

        # the client asks for IP address
        if msg_type == "DISCOVER":
            last_offered_ip = OFFERED_IP

            offer_message = {
                "type": "OFFER",
                "client_id": client_id,
                "ip": OFFERED_IP,
                "dns": DNS_SERVER_IP,
                "subnet_mask": SUBNET_MASK,
                "gateway": GATEWAY
            }

            # send the offer back to the client
            server_socket.sendto(json.dumps(offer_message).encode(), client_address)
            print(f"Sent OFFER to {client_id}")

        # the client asks to receive the offered IP
        elif msg_type == "REQUEST":
            requested_ip = message.get("ip")

            # make sure the client asks for the IP that was offered
            if requested_ip != last_offered_ip:
                error_message = {
                    "type": "ERROR",
                    "message": "Requested IP doesn't match the offered IP"
                }

                server_socket.sendto(json.dumps(error_message).encode(), client_address)
                print(f"Client {client_id} requested a different IP")
                continue

            ack_message = {
                "type": "ACK",
                "client_id": client_id,
                "ip": requested_ip,
                "dns": DNS_SERVER_IP,
                "subnet_mask": SUBNET_MASK,
                "gateway": GATEWAY
            }

            # ACK - server approves
            server_socket.sendto(json.dumps(ack_message).encode(), client_address)
            print(f"Sent ACK to {client_id}")

        else:
            print("Unknown message type")
```

DHCP_CLIENT

This file is the DHCP client, Its job is to ask the DHCP server for network settings.
The client creates a UDP socket and sends a DISCOVER message to the server to ask for an IP address. Then it waits for the server to send an OFFER with the IP and other configuration. After that, the client sends a REQUEST to confirm the IP it got. If the server sends back ACK, it means the configuration is approved. At the end, the client prints the IP, DNS, subnet mask, and gateway that it received.

```python
import socket
import json
from dhcp_config import DHCP_SERVER_HOST, DHCP_SERVER_PORT, BUFFER_SIZE


def main():
    # create a UDP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # address of the DHCP server
    server_address = (DHCP_SERVER_HOST, DHCP_SERVER_PORT)

    # simple id for this client
    client_id = "rami1"

    try:
        # send DISCOVER message
        discover_message = {
            "type": "DISCOVER",
            "client_id": client_id
        }

        client_socket.sendto(json.dumps(discover_message).encode(), server_address)
        print("Client sent DISCOVER")

        # receive the OFFER message from the server
        data, _ = client_socket.recvfrom(BUFFER_SIZE)
        offer_message = json.loads(data.decode())

        print("Client received:", offer_message)

        # if the server returns an error, stop
        if offer_message.get("type") == "ERROR":
            print("Server error:", offer_message.get("message"))
            return

        # check if server answered with OFFER
        if offer_message.get("type") != "OFFER":
            print("Error: server did not send OFFER")
            return

        offered_ip = offer_message.get("ip")

        # check if the server offered an IP address
        if offered_ip is None:
            print("Error: OFFER message doesn't contain an IP address")
            return

        # send REQUEST for the IP that was offered
        request_message = {
            "type": "REQUEST",
            "client_id": client_id,
            "ip": offered_ip
        }

        client_socket.sendto(json.dumps(request_message).encode(), server_address)
        print("Client sent REQUEST")

        # receive the answer from the server
        data, _ = client_socket.recvfrom(BUFFER_SIZE)
        ack_message = json.loads(data.decode())

        print("Client received:", ack_message)

        # if the server returns an error, stop
        if ack_message.get("type") == "ERROR":
            print("Server error:", ack_message.get("message"))
            return

        # ACK so the DHCP process finished successfully
        if ack_message.get("type") == "ACK":
            print("\nDHCP process completed successfully")
            print("Assigned IP:", ack_message.get("ip"))
            print("DNS Server:", ack_message.get("dns"))
            print("Subnet Mask:", ack_message.get("subnet_mask"))
            print("Default Gateway:", ack_message.get("gateway"))
        else:
            print("Error: DHCP process failed")

    except Exception as e:
        print("An error happened:", e)

    finally:
        client_socket.close()


if __name__ == "__main__":
    main()
```

DNS_CONFIG

This file contains the basic settings for the DNS part of the project.
It defines the server address, the port number, the buffer size, the domain name, and the IP address that should be returned for that domain.

I used this file to keep all the DNS constants in one place, so the code is cleaner and easier to change later if needed

```
DNS_SERVER_HOST = "127.0.0.1"
DNS_SERVER_PORT = 5556

# max number of bytes to recieve in one packet
BUFFER_SIZE = 1024

# the domain name that the client asks for
DOMAIN_NAME = "www.ariel.ac.il"

# the IP address returned for the domain
DOMAIN_IP = "127.0.0.1"
```

DNS_SERVER

This file implements the DNS server. The server waits for clients to send a domain name and returns the IP address for that domain. It creates a UDP socket, because DNS works over UDP, and binds to the host and port from the config file.

When a message arrives, the server checks if the client sent a DNS_QUERY.
If the domain name matches the one in the config file, the server sends back a DNS_RESPONSE with the IP address. If the domain is not found, the server sends an error message.

The server runs in a loop so it can answer multiple DNS requests.

```python
import socket
import json
from dns_config import DNS_SERVER_HOST, DNS_SERVER_PORT, BUFFER_SIZE, DOMAIN_NAME, DOMAIN_IP


def main():
    # create a UDP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # bind server to host and port
    server_socket.bind((DNS_SERVER_HOST, DNS_SERVER_PORT))

    print("DNS server started")
    print(f"Listening on {DNS_SERVER_HOST}:{DNS_SERVER_PORT}...")

    while True:
        # receive data from client
        data, client_address = server_socket.recvfrom(BUFFER_SIZE)

        try:
            # convert bytes to dictionary
            message = json.loads(data.decode())
        except json.JSONDecodeError:
            print("Received invalid JSON message")
            continue

        msg_type = message.get("type")
        domain = message.get("domain")

        print("Received from client:", message)

        # check if the client sent a domain name
        if domain is None:
            error_message = {
                "type": "DNS_ERROR",
                "message": "No domain was sent"
            }
            server_socket.sendto(json.dumps(error_message).encode(), client_address)
            print("Client didn't send a domain name")
            continue

        # client asks for the IP address of a domain
        if msg_type == "DNS_QUERY":
            if domain == DOMAIN_NAME:
                response_message = {
                    "type": "DNS_RESPONSE",
                    "domain": domain,
                    "ip": DOMAIN_IP
                }

                server_socket.sendto(json.dumps(response_message).encode(), client_address)
                print("Sent DNS response to client")

            else:
                error_message = {
                    "type": "DNS_ERROR",
                    "message": "Domain not found"
                }

                server_socket.sendto(json.dumps(error_message).encode(), client_address)
                print("Domain was not found")

        else:
            print("ERROR: Unknown message type")


if __name__ == "__main__":
    main()
```

DNS_CLIENT

This file implements the DNS client. The client sends a domain name to the DNS server and waits to get the IP address. First, the client creates a UDP socket and sends a DNS_QUERY message with the domain name. Then it waits for the server response. If the server sends DNS_RESPONSE, the client prints the IP address that was returned. If the server sends an error, the client prints the error message.

```python
import socket
import json
from dns_config import DNS_SERVER_HOST, DNS_SERVER_PORT, BUFFER_SIZE, DOMAIN_NAME


def main():
    # create a UDP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # address of the DNS server
    server_address = (DNS_SERVER_HOST, DNS_SERVER_PORT)

    try:
        # send DNS query for the domain name
        query_message = {
            "type": "DNS_QUERY",
            "domain": DOMAIN_NAME
        }

        client_socket.sendto(json.dumps(query_message).encode(), server_address)
        print("Client sent DNS query")

        # receive response from the server
        data, _ = client_socket.recvfrom(BUFFER_SIZE)
        response = json.loads(data.decode())

        print("Client received:", response)

        # check if the server returned an error
        if response.get("type") == "DNS_ERROR":
            print("DNS query failed")
            print("Reason:", response.get("message"))
            return

        # check if the server returned the IP address
        if response.get("type") == "DNS_RESPONSE":
            if response.get("ip") is None:
                print("DNS response does not contain an IP address")
                return
            print("\nDNS process completed successfully")
            print("Domain name:", response.get("domain"))
            print("IP address:", response.get("ip"))

        else:
            print("DNS query failed")
            print("Reason:", response.get("message"))

    except Exception as e:
        print("ERROR:", e)

    finally:
        client_socket.close()
```

APP_CONFIG

This file contains the main settings for the application part of the project. It defines the server address, the TCP port, the RUDP port, the buffer size, the folder of the files, the default file name, and also values like timeout and chunk size for RUDP.

```python
APP_SERVER_HOST = "127.0.0.1"
TCP_SERVER_PORT = 5557
RUDP_SERVER_PORT = 5558

# max number of bytes to recieve in one packet
BUFFER_SIZE = 1024

FILES_FOLDER = "APPLICATION/files"

DEFAULT_FILE_NAME = "index.html"

# timeout for waiting for ACK in RUDP
TIMEOUT = 2

# size of each file chunk sent by RUDP
CHUNK_SIZE = 512
```

TCP_SERVER

This file implements the TCP server that sends the file to the client using HTTP over TCP. The server creates a TCP socket, binds to the host and port, and starts listening for clients. When a client connects, the server receives an HTTP GET request that contains the file name. It checks if the file exists in the files folder. If the file exists, the server sends back a 200 OK response with the file content.
If the file does not exist, it sends 404 Not Found. After sending the response, the server closes the connection and waits for the next client.

```python
import socket
import os
from app_config import APP_SERVER_HOST, TCP_SERVER_PORT, BUFFER_SIZE, FILES_FOLDER


def main():
    # create a TCP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # bind server to host and port
    server_socket.bind((APP_SERVER_HOST, TCP_SERVER_PORT))

    server_socket.listen(5)
    print("TCP server started")
    print(f"Listening on {APP_SERVER_HOST}:{TCP_SERVER_PORT}...")

    while True:
        # accept connection from client
        client_socket, client_address = server_socket.accept()
        print("Client connected:", client_address)

        # receive request from client
        request = client_socket.recv(BUFFER_SIZE).decode()
        print("Received request:")
        print(request)

        # if nothing was received, close connection
        if not request:
            client_socket.close()
            print("Empty request!")
            continue

        # split the request into parts
        request_parts = request.split()

        # check if the request format is valid
        if len(request_parts) < 2 or request_parts[0] != "GET":
            response = "HTTP/1.1 400 Bad Request\r\n\r\nInvalid request"
            client_socket.send(response.encode())
            client_socket.close()
            print("Sent 400 Bad Request")
            continue

        # get the file name
        file_name = request_parts[1].lstrip("/")

        # build full path to the file
        file_path = os.path.join(FILES_FOLDER, file_name)

        # check if file exists
        if os.path.exists(file_path):
            with open(file_path, "r", encoding="utf-8") as file:
                file_data = file.read()

            response = "HTTP/1.1 200 OK\r\n\r\n" + file_data
            client_socket.send(response.encode())
            print(f"Sent file: {file_name}")

        else:
            response = "HTTP/1.1 404 Not Found\r\n\r\nFile not found"
            client_socket.send(response.encode())
            print(f"File not found: {file_name}")

        # close connection
        client_socket.close()
        print("Connection closed\n")


if __name__ == "__main__":
    main()
```

downloaded_index.html > html

```html
<html>
<head>
    <title>Networking Project</title>
</head>
<body>
    <h1>Welcome to my Networking Project</h1>
    <p>This file was sent by the application server.</p>

    <h2>Project Members</h2>
    <ul>
        <li>Rami</li>
    </ul>
</body>
</html>
```

This is the file content we get

TCP_CLIENT

This file implements the TCP client that downloads a file from the server.
The client creates a TCP socket and connects to the application server using the address and port from the config file. Then it sends a simple HTTP GET request with the file name. After that, the client waits for the server response and reads all the data until the connection is closed. The response is split into header and body.
If the server returned 200 OK, the client saves the file on the computer. If not, it prints an error message.

```python
def main():
    # create a TCP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # connect to the application server
    client_socket.connect((APP_SERVER_HOST, TCP_SERVER_PORT))
    print("Connected to TCP server")

    # build a HTTP GET request
    request = f"GET /{DEFAULT_FILE_NAME} HTTP/1.1\r\nHost: {APP_SERVER_HOST}\r\n\r\n"

    # send the request to the server
    client_socket.send(request.encode())
    print("Sent request to server")

    # receive the full response from the server
    response = b""
    while True:
        part = client_socket.recv(BUFFER_SIZE)
        if not part:
            break
        response += part

    # convert bytes to text
    response_text = response.decode()
    print("Received response from server:\n")
    print(response_text)

    # split the response into header and body (html parts)
    parts = response_text.split("\r\n\r\n", 1)

    if len(parts) < 2:
        print("Invalid response from server")
        client_socket.close()
        return

    header = parts[0]
    body = parts[1]

    # check if server returned the file successfully
    if "200 OK" in header:
        downloaded_file_name = "downloaded_" + DEFAULT_FILE_NAME

        # save the downloaded file
        with open(downloaded_file_name, "w", encoding="utf-8") as file:
            file.write(body)

        print(f"\nFile downloaded successfully: {downloaded_file_name}")
    else:
        print("\nThe server didn't return the file successfully")
```

RUDP_SERVER

This file implements the server for Reliable UDP. Since UDP is not reliable, I added my own logic to make sure the file is sent correctly. The server creates a UDP socket and waits for the client to send a request for a file. When the request arrives, the server reads the file and splits it into small chunks. Each chunk is sent with a sequence number. After sending each packet, the server waits for an ACK from the client. If the ACK is not received before timeout, the server sends the packet again. At the end, the server sends a last packet to mark that the file transfer is finished.

```python
from app_config import (
    APP_SERVER_HOST,
    RUDP_SERVER_PORT,
    BUFFER_SIZE,
    FILES_FOLDER,
    TIMEOUT,
    CHUNK_SIZE
)


def main():
    # create a UDP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # bind server to host and port
    server_socket.bind((APP_SERVER_HOST, RUDP_SERVER_PORT))

    # set timeout for waiting for ACK messages
    server_socket.settimeout(TIMEOUT)

    print("RUDP server started")
    print(f"Listening on {APP_SERVER_HOST}:{RUDP_SERVER_PORT}...")

    while True:
        try:
            # receive request from client
            data, client_address = server_socket.recvfrom(BUFFER_SIZE)
        except socket.timeout:
            # no request arrived yet
            continue

        try:
            # convert bytes to dictionary
            message = json.loads(data.decode())
        except json.JSONDecodeError:
            print("Received invalid JSON message")
            continue

        print("Received from client:", message)

        # client asks to download a file
        if message.get("type") == "GET_FILE":
            file_name = message.get("file_name")
            file_path = os.path.join(FILES_FOLDER, file_name)

            # check if the file exists
            if not os.path.exists(file_path):
                error_message = {
                    "type": "ERROR",
                    "message": "File not found"
                }
```

```python
            server_socket.sendto(json.dumps(error_message).encode(), client_address)
            print("File not found")
            continue

        # read the file content
        with open(file_path, "r", encoding="utf-8") as file:
            file_data = file.read()

        # split file into smaller chunks
        chunks = []
        for i in range(0, len(file_data), CHUNK_SIZE):
            chunks.append(file_data[i:i + CHUNK_SIZE])

        seq = 0

        # stop and wait: send each chunk and wait for ACK before sending the next one
        for chunk in chunks:
            packet = {
                "type": "DATA",
                "seq": seq,
                "data": chunk,
                "last": False
            }

            while True:
                if random.random() < 0.2:
                    print(f"Simulated loss of packet {seq}")
                else:
                    server_socket.sendto(json.dumps(packet).encode(), client_address)
                    print(f"Sent packet {seq}")

                try:
                    ack_data, _ = server_socket.recvfrom(BUFFER_SIZE)
                    ack_message = json.loads(ack_data.decode())

                    if ack_message.get("type") == "ACK" and ack_message.get("seq") == seq:
                        print(f"Received ACK for packet {seq}")
                        break

                except socket.timeout:
                    print(f"Timeout on packet {seq}, sending again...")

            seq += 1
```

```python
        # send final packet (file finished))
        end_packet = {
            "type": "DATA",
            "seq": seq,
            "data": "",
            "last": True
        }

        while True:
            if random.random() < 0.2:
                    print(f"Simulated loss of last packet {seq}")
            else:
                server_socket.sendto(json.dumps(end_packet).encode(), client_address)
                print("Sent last packet")

            try:
                ack_data, _ = server_socket.recvfrom(BUFFER_SIZE)
                ack_message = json.loads(ack_data.decode())

                if ack_message.get("type") == "ACK" and ack_message.get("seq") == seq:
                    print("Received ACK for last packet")
                    break

            except socket.timeout:
                print("Timeout on last packet, sending again...")
```

RUDP_CLIENT

This file implements the client for Reliable UDP. The client asks the server for a file and receives it in small packets.
First, the client sends a request with the file name. Then it waits to receive packets from the server. Each packet has a sequence number, so the client checks that the packets arrive in the correct order.
After receiving a packet, the client sends an ACK to the server to confirm it was received. If a packet arrives with the wrong number, the client asks for the last correct packet again.
All the data is saved in order, and when the last packet arrives, the client writes the file to disk.

```python
def main():
    # create UDP socket for the client
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # address of the RUDP server
    server_address = (APP_SERVER_HOST, RUDP_SERVER_PORT)

    # this will keep the file data in the correct order
    full_file_data = ""

    # the client expects packets in order: 0, 1, 2, ...
    expected_seq = 0

    try:
        # send request to download the file
        request_message = {
            "type": "GET_FILE",
            "file_name": DEFAULT_FILE_NAME
        }

        client_socket.sendto(json.dumps(request_message).encode(), server_ad
        print("Client sent file request")

        while True:
            # receive packet from server
            data, _ = client_socket.recvfrom(BUFFER_SIZE)
            message = json.loads(data.decode())

            print("Client received:", message)

            # check if server returned an error
            if message.get("type") == "ERROR":
                print("Server error:", message.get("message"))
                break

            # process file data packets
            if message.get("type") == "DATA":
                seq = message.get("seq")
                chunk_data = message.get("data")
                is_last = message.get("last")

                # accept only the expected packet number
                if seq == expected_seq:
                    full_file_data += chunk_data

                    ack_message = {
                        "type": "ACK",
                        "seq": seq
                    }

                    client_socket.sendto(json.dumps(ack_message).encode(), server_address)
                    print(f"Sent ACK for packet {seq}")

                    expected_seq += 1

                    # if this is the last packet, stop receiving
                    if is_last:
                        print("Received last packet")
                        break

                else:
                    # send ACK again for the last correct packet if needed
                    ack_message = {
                        "type": "ACK",
                        "seq": expected_seq - 1
                    }

                    client_socket.sendto(json.dumps(ack_message).encode(), server_address)
                    print("Received unexpected packet number")

    # save the downloaded file
    downloaded_file_name = "downloaded_" + DEFAULT_FILE_NAME
    with open(downloaded_file_name, "w", encoding="utf-8") as file:
        file.write(full_file_data)

    print(f"\nFile downloaded successfully: {downloaded_file_name}")

except Exception as e:
    print("An error happened:", e)

finally:
    client_socket.close()
    print("Connection closed")
```

MAIN_CLIENT.PY

FINALLY EVERYTHING WORKS TOGETHER

After writing many servers and clients, I made one main client that connects all parts.

MAIN_CLIENT

```python
def run_dhcp():
    # create a UDP socket for DHCP
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    server_address = (DHCP_SERVER_HOST, DHCP_SERVER_PORT)

    try:
        # send DISCOVER message to DHCP server
        discover_message = {
            "type": "DISCOVER",
            "client_id": "main_client"
        }

        client_socket.sendto(json.dumps(discover_message).encode(), server_address)
        print("\n[DHCP] Sent DISCOVER")

        # receive OFFER from server
        data, _ = client_socket.recvfrom(DHCP_BUFFER)
        offer_message = json.loads(data.decode())

        print("[DHCP] Received:", offer_message)

        if offer_message.get("type") != "OFFER":
            print("[DHCP] Error: server did not send OFFER")
            return None

        offered_ip = offer_message.get("ip")

        # send REQUEST for the offered IP
        request_message = {
            "type": "REQUEST",
            "client_id": "main_client",
            "ip": offered_ip
        }

        client_socket.sendto(json.dumps(request_message).encode(), server_address)
        print("[DHCP] Sent REQUEST")

        # receive ACK from server
        data, _ = client_socket.recvfrom(DHCP_BUFFER)
        ack_message = json.loads(data.decode())

        print("[DHCP] Received:", ack_message)

        # stop if server returned an error
        if ack_message.get("type") == "ERROR":
            print("[DHCP] Server error:", ack_message.get("message"))
            return None

        if ack_message.get("type") == "ACK":
            print("[DHCP] Configuration completed successfully")

            return {
                "ip": ack_message.get("ip"),
                "dns": ack_message.get("dns"),
                "subnet_mask": ack_message.get("subnet_mask"),
                "gateway": ack_message.get("gateway")
            }

        else:
            print("[DHCP] ERROR: DHCP process failed")
            return None

    except Exception as e:
        print("[DHCP] ERROR:", e)
        return None

    finally:
        client_socket.close()


def run_dns():
    # create a UDP socket for DNS
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    server_address = (DNS_SERVER_HOST, DNS_SERVER_PORT)

    try:
        # send DNS query for the domain name
        query_message = {
            "type": "DNS_QUERY",
            "domain": DOMAIN_NAME
        }

        client_socket.sendto(json.dumps(query_message).encode(), server_address)
        print("\n[DNS] Sent DNS query")

        # receive response from DNS server
        data, _ = client_socket.recvfrom(DNS_BUFFER)
        response = json.loads(data.decode())

        print("[DNS] Received:", response)

        if response.get("type") == "DNS_ERROR":
            print("[DNS] DNS query failed")
            print("[DNS] Reason:", response.get("message"))
            return None

        if response.get("type") == "DNS_RESPONSE":
            print("[DNS] DNS process completed successfully")
            return response.get("ip")
        else:
            print("[DNS] DNS query failed")
            print("[DNS] Reason:", response.get("message"))
            return None

    except Exception as e:
        print("[DNS] ERROR:", e)
        return None

    finally:
        client_socket.close()


def download_with_tcp(server_ip):
    # create a TCP socket for the client
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        # connect to the TCP application server
        client_socket.connect((server_ip, TCP_SERVER_PORT))
        print(f"\n[TCP] Connected to server {server_ip}:{TCP_SERVER_PORT}")

        # build a HTTP GET request
        request = f"GET /{DEFAULT_FILE_NAME} HTTP/1.1\r\nHost: {server_ip}\r\n\r\n"

        # send the request to the server
        client_socket.send(request.encode())
        print("[TCP] Sent request to server")

        # receive the full response from the server
        response = b""
        while True:
            part = client_socket.recv(APP_BUFFER)
            if not part:
                break
            response += part

        # convert bytes to text
        response_text = response.decode()
        print("[TCP] Received response from server")

        # split the response into header and body (html parts)
        parts = response_text.split("\r\n\r\n", 1)

```

MAIN_CLIENT

```python
        if len(parts) < 2:
            print("[TCP] Invalid response from server")
            return

        header = parts[0]
        body = parts[1]

        # check if server returned the file successfully
        if "200 OK" in header:
            downloaded_file_name = "downloaded_tcp_" + DEFAULT_FILE_NAME

            # save the downloaded file
            with open(downloaded_file_name, "w", encoding="utf-8") as file:
                file.write(body)

            print(f"[TCP] File downloaded successfully: {downloaded_file_name}")
        else:
            print("[TCP] The server didn't return the file successfully")

    except Exception as e:
        print("[TCP] ERROR:", e)

    finally:
        client_socket.close()
        print("[TCP] Connection closed")


def download_with_rudp(server_ip):
    # create a UDP socket for the client
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    server_address = (server_ip, RUDP_SERVER_PORT)

    # this will keep the file data in the correct order
    full_file_data = ""

    # the client expects packets in order: 0, 1, 2, ...
    expected_seq = 0

    download_success = False

    try:
        # send request to download the file
        request_message = {
            "type": "GET_FILE",
            "file_name": DEFAULT_FILE_NAME
        }

        client_socket.sendto(json.dumps(request_message).encode(), server_address)
        print(f"\n[RUDP] Sent file request to {server_ip}:{RUDP_SERVER_PORT}")
```

```python
        while True:
            # receive packet from server
            data, _ = client_socket.recvfrom(APP_BUFFER)
            message = json.loads(data.decode())

            print("[RUDP] Received:", message)

            # check if server returned an error
            if message.get("type") == "ERROR":
                print("[RUDP] Server error:", message.get("message"))
                return

            # process data packets
            if message.get("type") == "DATA":
                seq = message.get("seq")
                chunk_data = message.get("data")
                is_last = message.get("last")

                # accept only the expected packet number
                if seq == expected_seq:
                    full_file_data += chunk_data

                    ack_message = {
                        "type": "ACK",
                        "seq": seq
                    }

                    client_socket.sendto(json.dumps(ack_message).encode(), server_address)
                    print(f"[RUDP] Sent ACK for packet {seq}")

                    expected_seq += 1

                    # if this is the last packet, stop receiving
                    if is_last:
                        print("[RUDP] Received last packet")
                        download_success = True
                        break

                else:
                    # send ACK again for the last correct packet
                    ack_message = {
                        "type": "ACK",
                        "seq": expected_seq - 1
                    }

                    client_socket.sendto(json.dumps(ack_message).encode(), server_address)
                    print("[RUDP] Received unexpected packet number")
```

```python
def main():
    print("Choose how you want to download the file:")
    print("1 - TCP")
    print("2 - Reliable UDP")

    choice = input("Enter your choice: ")

    if choice == "1":
        print("You chose TCP")
    elif choice == "2":
        print("You chose Reliable UDP")
    else:
        print("Invalid choice")
        return

    # 1) get network configuration from DHCP
    dhcp_data = run_dhcp()

    if dhcp_data is None:
        print("Couldn't continue because DHCP failed")
        return

    print("\nClient network configuration:")
    print("Client IP:", dhcp_data["ip"])
    print("DNS Server:", dhcp_data["dns"])
    print("Subnet Mask:", dhcp_data["subnet_mask"])
    print("Default Gateway:", dhcp_data["gateway"])

    # 2) ask the DNS server for the website IP
    server_ip = run_dns()

    if server_ip is None:
        print("Couldn't continue because DNS failed")
        return

    print("\nThe domain was resolved successfully")
    print("Domain:", DOMAIN_NAME)
    print("Server IP:", server_ip)

    # 3) download the file using the chosen protocol
    if choice == "1":
        download_with_tcp(server_ip)
    else:
        download_with_rudp(server_ip)
```

MAIN_CLIENT

The main client connects all parts of the project together.
Instead of running DHCP, DNS, TCP, and RUDP separately, this client does the whole process in one program.
At the beginning, the user chooses how to download the file — using TCP or Reliable UDP, Then the client starts the network process step by step.

First, it contacts the DHCP server to get network configuration like IP address, DNS, subnet mask, and gateway,
After that, it contacts the DNS server to get the IP address of the application server from the domain name.
When the client knows the server IP, it downloads the file using the protocol that he selected.
If TCP is chosen, the client sends an HTTP GET request and receives the file.
If RUDP is chosen, the client receives the file in packets, sends ACK for each packet, and saves the file at the end. I wrote this file to show that the whole system works together in the correct order:

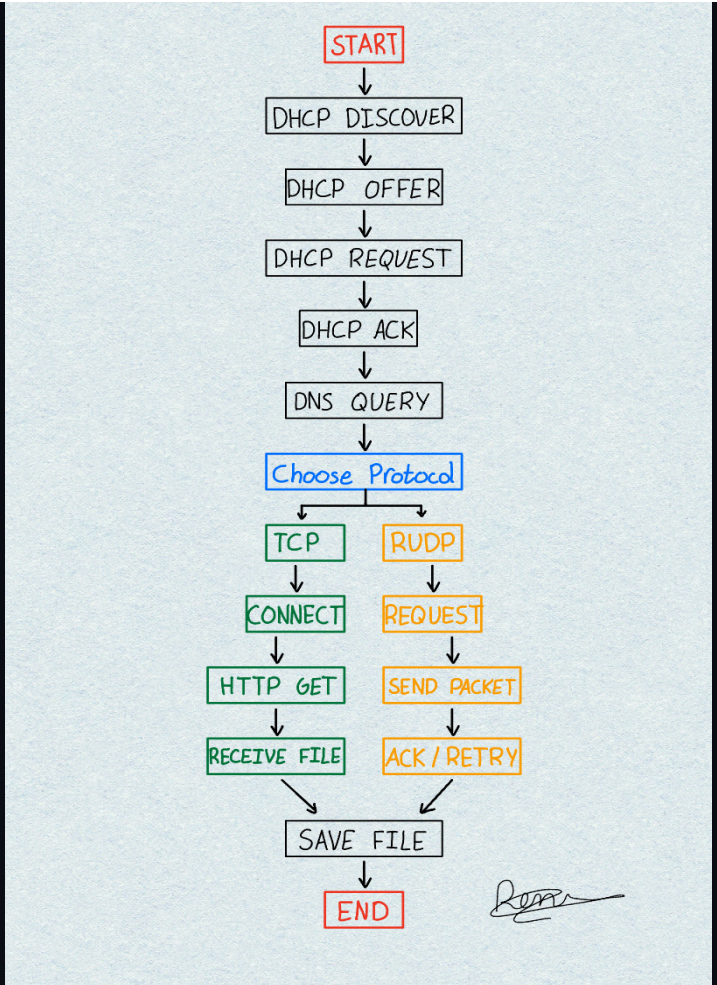
DHCP → DNS → TCP / RUDP → file download

How I used ChatGPT during the project

1. Explain how DHCP works step by step
2. Explain reliable UDP and how can I implement it using python
3. What port numbers can I use for DHCP, DNS, HTTP?
4. How to make a packet loss in RUDP, look at my program and tell me if I have to add it here
5. Explain about congestion control and flow control
6. How to split a file into chunks before I send them over UDP?
7. What should a HTTP GET request look like?
8. How to check if my TCP server works correctly?
9. What should i show in Wireshark to prove that my code works?
10. How to draw a state diagram for RUDP
11. Write for me a simple html file that I can use it in the project

...

STATE DIAGRAM



HOW DOES THE SYSTEM HANDLE PACKET LOSS?

In my system, packet loss is handled in the RUDP part of the project.
Since UDP does not guarantee delivery, i implemented my own reliability. Each packet is sent with a sequence number, so the client knows which packet it should receive next. After receiving a packet, the client sends an ACK message back to the server. The server waits for the ACK, and if it does not receive it within a timeout, it assumes the packet was lost. When a timeout happens, the server sends the same packet again. This continues until the correct ACK is received. I also use a stop and wait method, which means the server sends one packet at a time and waits for an ACK before sending the next one. This helps keep the packets in order and prevents errors.
Because of this mechanism, even if packets are lost, the system can resend them and the file is received correctly.

HOW DOES THE SYSTEM HANDLE LATENCY PROBLEMS?

In my system, latency problems are handled using timeouts and acknowledgments, especially in the RUDP part, Because UDP does not guarantee delivery and packets may arrive late, the server waits for an ACK after sending each packet. If the ACK does not arrive within a certain timeout, the server assumes that the packet was delayed or lost, and sends it again.
This timeout mechanism allows the system to continue working even when there is network delay.
In the TCP mode, latency is handled automatically by TCP, since TCP already has built-in mechanisms for retransmission, acknowledgments, and flow control.
Because of these mechanisms, the system can still download the file correctly even when there is delay in the network.

כיצד הייתי מרחיב את המערכת?

אם הייתי רוצה להרחיב את המערכת, הייתי מוסיף דברים כדי שתעבוד בצורה יותר אמיתית ויותר מתקדמת. קודם כל, השרתים שלי עובדים בצורה פשוטה וכל בקשה מטופלת בנפרד, אבל כדי לאפשר עבודה עם מספר רב של לקוחות במקביל הייתי משפר את זה למשל באמצעות Async .

בנוסף, אפשר לשפר את ה RUDP ולהוסיף Sliding Window כדי לשלוח כמה חבילות ברצף, זה יעזור כשיש latency גבוה או הרבה מידע לשלוח.

ב DNS אפשר להוסיף תמיכה בכמה דומיינים.

בנוסף, אפשר להוסיף בדיקות אבטחה, למשל לבדוק מי מתחבר לשרת, או להגביל גישה לקבצים.

בסה"כ המערכת שלי היא רק בסיס, ואפשר להרחיב אותה למערכת רשת יותר אמיתית שיכולה לעבוד עם הרבה משתמשים בו זמנית.